

Final Exercise SQL

Beatriz Sánchez Espada

1. Given an SQLite database containing IMDB data composed of three tables: imdb, earning, and genre. Implement a Python program to display the following information:

The complete python script of the two exercises is found within the report.

Before going with each exercise, we must import the sqlite library to manage the database using import sqlite3, next we also import matplotlib package to make the different plots. Once we have this, we also have to connect to the database and create a cursor to execute the SQL commands.

```
import sqlite3 # Import the SQLite library to manage the database
import matplotlib.pyplot as plt

# Connect to SQLite database (will create it if it doesn't exist)
conn = sqlite3.connect('IMDB-movies.sqlite')
cursor = conn.cursor() # Create a cursor to execute SQL commands
```

Top 10 highest-rated movies.

For extracting the top 10 highest rated movies on the IMDB database we have to make the following code:

```
def top_10():
    cursor.execute('''SELECT Title, Rating FROM IMDB ORDER BY Rating DESC LIMIT 10''')
    results = cursor.fetchall()

    print("Top 10 Highest-Rated Movies:\n")
    for title, rating in results:
        print(f"{title} - Rating: {rating}")
```

We first execute an SQL query that selects the Title and Rating columns from the database, sorting the results in descending order based on rating and limiting the output to 10 movies. We then store the fetched data in the results variable using cursor.fetchall(), which retrieves all matching rows. Finally, we print the header "Top 10 Highest-Rated Movies:\n" and iterate through the results list using a for loop, formatting and displaying each movie's title alongside its rating.

After executing the code, it gives the following output showing the highest rated movie as Inception (2010).

```
Top 10 Highest-Rated Movies:

Inception (2010) - Rating: 8.8
Interstellar (2014) - Rating: 8.6
Whiplash (2014) - Rating: 8.5
Django Unchained (2012) - Rating: 8.4
The Dark Knight Rises (2012) - Rating: 8.4
Toy Story 3 (2010) - Rating: 8.3
Hacksaw Ridge (2016) - Rating: 8.2
Inside Out (2015) - Rating: 8.2
La La Land (2016) - Rating: 8.2
The Wolf of Wall Street (2013) - Rating: 8.2
```

Rating of the highest-grossing movie worldwide.

```
def highest_grossing():
    cursor.execute('''SELECT IMDB.Title, IMDB.Rating, earning.Worldwide
                      FROM IMDB
                      INNER JOIN earning ON earning.Movie_id = IMDB.Movie_id
                      ORDER BY earning.Worldwide DESC
                      LIMIT 1;''')
    results = cursor.fetchall()

    print("\nRating of Highest-Grossing Movie Worldwide:")
    for title, rating, revenue in results:
        print(f"{title} - Rating: {rating} Worldwide Gross: ${revenue:,}")
```

We execute an SQL query that selects the Title and Rating columns from the IMDB table along with the Worldwide earnings from the earning table. To combine data from both tables, we use an INNER JOIN on Movie_id. We then sort the results in descending order based on worldwide earnings, ensuring that the highest-grossing movie appears first. The query is limited to one movie using LIMIT 1.

Next, we fetch the results using cursor.fetchall(), print a header, and iterate through the retrieved data. Finally, we display the movie's title and worldwide gross revenue, formatting the revenue with commas for better readability.

With this code the Rating of the highest grossing movie is Star Wars:The Force Awakens with a Rating of 8.1 and a grossing of more than 2 billion dollars.

Rating of Highest-Grossing Movie Worldwide:
Star Wars: The Force Awakens (2015) - Rating: 8.1 Worldwide Gross: \$2,068,223,624.0

Distribution of movie ratings.

For showing the distribution of the ratings of the different movies on the database we can use the following code:

```
def distribution():
    cursor.execute('''SELECT Rating, COUNT(*) as movie_count
                      FROM IMDB
                      GROUP BY Rating
                      ORDER BY Rating DESC''')

    results = cursor.fetchall()

    ratings = [row[0] for row in results]
    movie_count = [row[1] for row in results]

    ratings = list(map(str, ratings))

    plt.figure(figsize=(10, 5))
    plt.bar(ratings, movie_count, color="skyblue")

    plt.xlabel('Rating')
    plt.ylabel('Number of Movies')
    plt.title('Distribution of Movies by Rating')
    plt.grid(axis="y", linestyle="--", alpha=0.7)

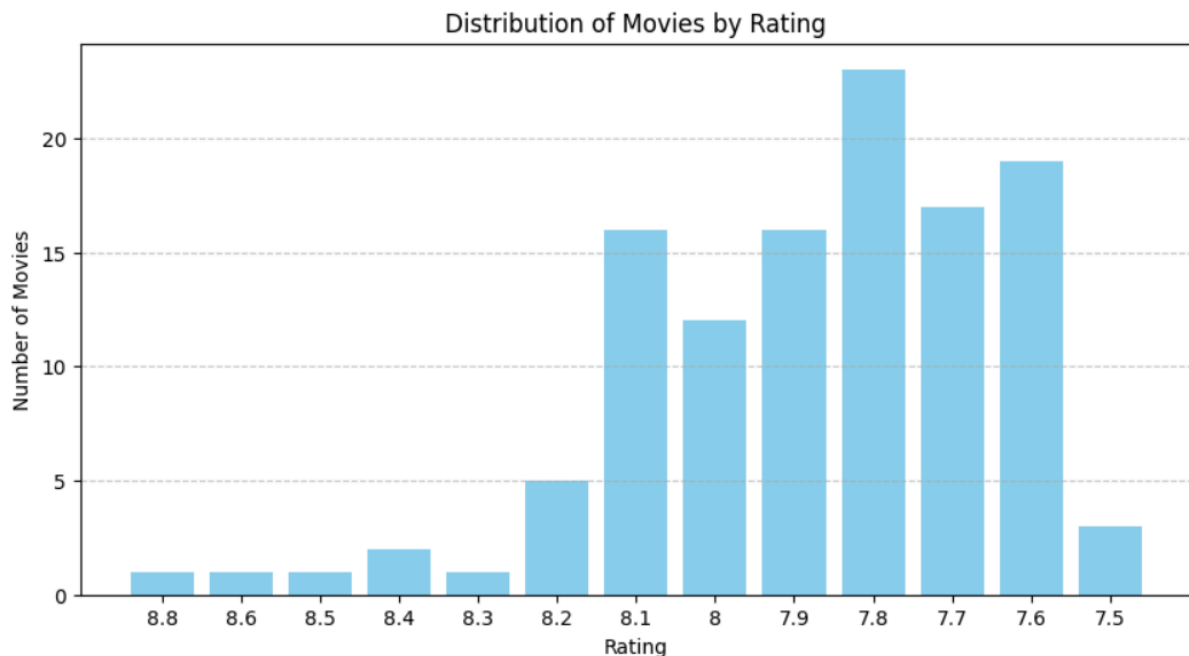
    plt.show()
```

We execute an SQL query that selects the Rating and the count of movies (movie_count) from the IMDB table. The query groups the results by Rating, counting the number of movies for each rating. The results are then sorted in descending order based on the rating.

Next, we fetch the data using cursor.fetchall() and extract the ratings and corresponding movie counts into separate lists. Since ratings might be numeric, we convert them to strings for labeling purposes.

We then create a bar chart using matplotlib.pyplot. The x-axis represents the ratings, while the y-axis represents the number of movies with that rating. The bars are colored "skyblue", and a grid is added to the y-axis for better visualization. Finally, we display the chart using plt.show().

The plot shows an increase of the quantity of movies with a rating between 8.1 and 7.6, having less than 5 movies per rating when it's higher than 8.3. In fact, using the information provided by the exercise one, we know that the movies that surpass the 8.3 in rating in this database are The Dark Knight Rises, Django, Whiplash, Interstellar and Inception.



Movie name and running time of the longest-running movie.

```
def longest_running():
    cursor.execute('''SELECT Title, Runtime
                      FROM IMDB
                      ORDER BY Runtime DESC
                      LIMIT 1;''')
    results = cursor.fetchall()

    print("\nLongest-Running Movie:")
    for title, runtime in results:
        print(f"{title} - Runtime: {runtime} minutes")
```

We execute an SQL query that selects the Title and Runtime columns from the IMDB table, sorting the results in descending order based on runtime. By applying LIMIT 1, we ensure that only the longest-running movie is retrieved.

Next, we fetch the results using cursor.fetchall(), print a header, and iterate through the retrieved data. Finally, we display the movie title along with its runtime in minutes for clear and formatted output.

Once we execute the code the result is the following:

```
Longest-Running Movie:  
The Grand Budapest Hotel (2014) - Runtime: 99 min minutes
```

Movie with the highest net profit.

For finding the movie with the highest net profit we use the following code:

```
def net_profit():  
    cursor.execute('''SELECT IMDB.Title,  
                        (earning.Worldwide - IMDB.Budget) AS net_profit  
                        FROM IMDB  
                        INNER JOIN earning ON earning.Movie_id = IMDB.Movie_id  
                        ORDER BY net_profit DESC  
                        LIMIT 1;''')  
    results = cursor.fetchall()  
  
    print("\nMovie with the Highest Net Profit:")  
    for title, profit in results:  
        print(f"{title} - Net Profit: ${profit:,}")
```

We execute an SQL query that selects the Title from the IMDB table and calculates the net profit by subtracting the Budget from the Worldwide earnings in the earning table. INNER JOIN is used on Movie_id to combine data from both tables. The results are then sorted in descending order based on net profit, ensuring that the most profitable movie appears first. The query is limited to one movie using LIMIT 1.

Next, we fetch the results using cursor.fetchall(), print a header, and iterate through the retrieved data. Finally, we display the movie title and net profit, formatting the profit with commas for better readability.

According to the database, the movie with the highest net profit is Star Wars: The Force Awakens with a profit of more than \$1 billion.

```
Movie with the Highest Net Profit:  
Star Wars: The Force Awakens (2015) - Net Profit: $1,823,223,624.0
```

Least-budgeted movie among those with the highest net profit.

```
def least_budgeted():
    cursor.execute('''SELECT IMDB.Title, IMDB.Budget,
                          (earning.Worldwide - IMDB.Budget) AS net_profit
                        FROM IMDB
                        INNER JOIN earning ON earning.Movie_id = IMDB.Movie_id
                        WHERE net_profit > 500000000
                        ORDER BY IMDB.Budget ASC LIMIT 1''')
    results = cursor.fetchall()

    print("\nLeast-Budgeted Movie Among the Highest Net-Profit Movies:")
    for title, budget, profit in results:
        print(f"{title} - Budget: ${budget:,}, Net Profit: ${profit:,}")
```

We execute an SQL query that selects the Title and Budget from the IMDB table, and calculate the Net Profit Worldwide - Budget. Using an INNER JOIN on Movie_id, we combine data from both the IMDB and earning tables.

We filter the results to include only movies with a net profit greater than \$500 million ($\text{net_profit} > 500000000$), ensuring we focus on the most profitable films. Then, we order the results in ascending order based on Budget (IMDB.Budget ASC) to find the movie with the lowest budget among those profitable films. The query is limited to one movie using LIMIT 1.

After retrieving the results with cursor.fetchall(), we print a header and loop through the data. We display the movie title, budget, and net profit, formatting the numbers with commas for improved readability.

According to the output, *Deadpool* has the lowest budget. However, this result is entirely dependent on the chosen net profit range. In other words, if the range were narrower, for instance, considering only movies with over \$1 billion in profit instead of \$500 million, the outcome could be different.

```
Least-Budgeted Movie Among the Highest Net-Profit Movies:
Deadpool (2016) - Budget: $58,000,000.0, Net Profit: $725,112,979.0
```

Percentage of the total budget allocated to each genre, visualized as a bar chart.

```
def percentage_budget():
    cursor.execute('''SELECT genre.Genre, SUM(IMDB.Budget) AS genre_budget
                      FROM IMDB
                      INNER JOIN genre ON IMDB.Movie_id = genre.Movie_id
                      GROUP BY genre.Genre;''')

    results = cursor.fetchall()

    cursor.execute('SELECT SUM(Budget) FROM IMDB')
    total_budget = cursor.fetchone()[0]

    genres = [row[0] for row in results]
    genre_budget = [row[1] for row in results]
    percentage_budget = [(budget / total_budget) * 100 for budget in genre_budget]

    plt.figure(figsize=(12, 6))
    plt.bar(genres, percentage_budget, color="green")

    plt.xlabel('Genre')
    plt.ylabel('Percentage of Total Budget (%)')
    plt.title('Percentage of Total Budget Allocated to Each Genre')

    plt.xticks(rotation=45, ha='right')
    plt.grid(axis="y", linestyle="--", alpha=0.7)
    plt.tight_layout()

    plt.show()
```

We execute an SQL query that selects the Genre from the genre table and the sum of the budgets for each genre from the IMDB table. We use an INNER JOIN on Movie_id to combine data from both tables and group the results by genre. This query gives the total budget allocated to each genre.

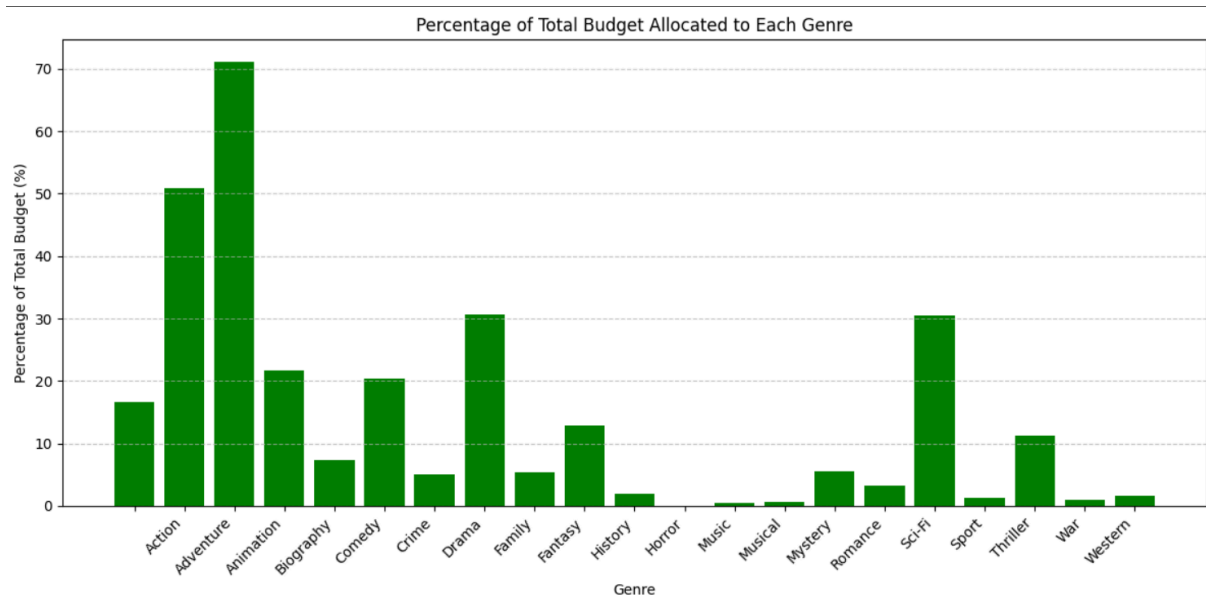
Next, we execute another query to fetch the total budget of all movies in the IMDB table by summing the Budget column.

We then create two lists: one for the genres and another for the total budget per genre. Using this data, we calculate the percentage of the total budget allocated to each genre by dividing the genre's budget by the total budget and multiplying by 100.

The data is displayed using a bar chart where the x-axis represents the genres, and the y-axis represents the percentage of the total budget allocated to each genre. The chart is customized with green bars, rotated x-axis labels for better readability, and a grid for improved visualization. Finally, we use `plt.tight_layout()` to adjust the layout and ensure the labels are not cut off.

According to the output, the genres with the highest percentage of budget allocation are *Adventure* and *Action*, followed by *Sci-Fi* and *Drama*. In contrast, the genres with the lowest budget percentages are *Horror*, *Musical*, and *Music*. One possible explanation for this is that

Adventure, Action, and Sci-Fi films often require expensive resources such as CGI and VFX effects, which can significantly increase production costs.



Movies that belong to both the Adventure and Action genres and have a rating of 7 or higher.

For finding the movies belonging to Adventure and Action we can execute the following code:

```
def both_genres():
    cursor.execute('''SELECT IMDB.Title,IMDB.Rating
                      FROM IMDB
                      INNER JOIN genre ON genre.Movie_id = IMDB.Movie_id
                      WHERE genre.genre IN ("Action", "Adventure")
                      AND IMDB.Rating >= 7
                      GROUP BY IMDB.Movie_id
                      HAVING COUNT(DISTINCT genre.genre) = 2
                      ORDER BY IMDB.Rating DESC;''')
    results = cursor.fetchall()

    print("\nMovies Belonging to Both Action and Adventure Genres (Rating >= 7):\n")
    for Title,rating in results:
        print(f"{Title} - Rating: {rating}")
```

We execute an SQL query that selects the Title and Rating from the IMDB table for movies that belong to both the Action and Adventure genres. The query uses an INNER JOIN on the Movie_id field to combine data from both the IMDB and genre tables.

The following query filters the results to include movies that belong to both genres (genre.genre IN ("Action", "Adventure")) and have a rating greater than or equal to 7 (IMDB.Rating >= 7). The GROUP BY clause groups the results by Movie_id, and the HAVING clause ensures that only movies that have both genres are included. The results are then sorted in descending order based on Rating.

After fetching the results with cursor.fetchall(), we print a header and then iterate through the retrieved data. Each movie that meets the criteria is displayed by its title and rating.

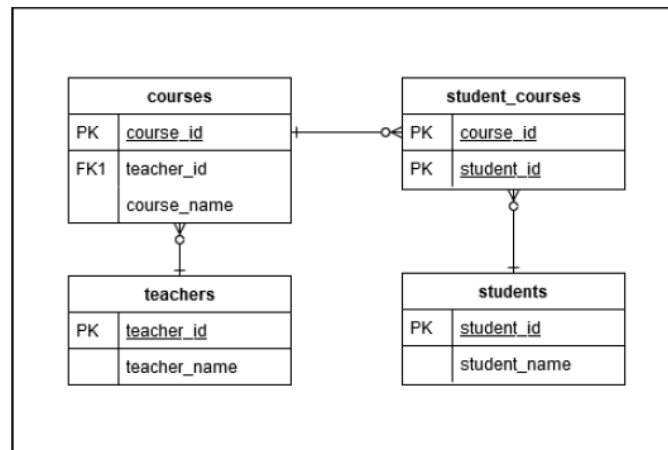
This output shows that in this database there are 21 Adventure and Action movies with a rating greater than 7.

```
Movies Belonging to Both Action and Adventure Genres (Rating >= 7):
```

```
Inception (2010) - Rating: 8.8
How to Train Your Dragon (2010) - Rating: 8.1
Star Wars: The Force Awakens (2015) - Rating: 8.1
Mad Max: Fury Road (2015) - Rating: 8.1
Guardians of the Galaxy (2014) - Rating: 8.1
X-Men: Days of Future Past (2014) - Rating: 8
Deadpool (2016) - Rating: 8
Captain America: Civil War (2016) - Rating: 7.9
Rogue One (2016) - Rating: 7.9
How to Train Your Dragon 2 (2014) - Rating: 7.9
Edge of Tomorrow (2014) - Rating: 7.9
Skyfall (2012) - Rating: 7.8
The Lego Movie (2014) - Rating: 7.8
X-Men: First Class (2011) - Rating: 7.8
Star Trek Into Darkness (2013) - Rating: 7.8
Captain America: The Winter Soldier (2014) - Rating: 7.8
Big Hero 6 (2014) - Rating: 7.8
Kingsman: The Secret Service (2014) - Rating: 7.7
The Hunger Games: Catching Fire (2013) - Rating: 7.6
Doctor Strange (2016) - Rating: 7.6
Dawn of the Planet of the Apes (2014) - Rating: 7.6
```

2. REVIEW EXERCISE

Create a python program to handle the data from students and subjects. For that, create a simple database with the following entity-relationship diagram.



The program will allow inserting courses and students. Moreover, the program will allow enrolling students in courses. Then, for a given student, the program will report the list of subjects he/she is enrolled in.

```
import sqlite3

def create_database():
    with sqlite3.connect("school.db") as connection:
        cursor = connection.cursor()
```

We connect to the SQLite database using `sqlite3.connect("school.db")`, which either creates the database or opens it if it already exists. We store the connection in the variable `connection` and create a cursor with `connection.cursor()` to execute SQL commands. Then we can begin creating the tables.

```

# Create tables
cursor.execute("""
CREATE TABLE IF NOT EXISTS teachers (
    teacher_id INTEGER PRIMARY KEY,
    teacher_name TEXT NOT NULL
);
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS courses (
    course_id INTEGER PRIMARY KEY,
    teacher_id INTEGER,
    course_name TEXT NOT NULL,
    FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)
);
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT NOT NULL
);
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS student_courses (
    course_id INTEGER,
    student_id INTEGER,
    PRIMARY KEY (course_id, student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
""")

```

Creating the teachers table:

We define the teachers table with two columns: teacher_id (which serves as the primary key) and teacher_name.

Creating the courses table:

We create the courses table, which includes a course_id (primary key), teacher_id (foreign key referencing teacher_id in the teachers table), and course_name to store the name of each course.

Creating the students table:

We define the students table with student_id (primary key) and student_name to store the names of the students.

Creating the student_courses table:

The student_courses table links students to courses. It has a composite primary key (course_id, student_id), ensuring that a student can only be enrolled in a specific course once. The foreign keys reference the courses and students tables.

```
connection.commit()
connection.close()
```

After creating the tables, we use connection.commit() to save the changes to the database and then close the connection with connection.close().

```
def insert_data(table, **values):
    with sqlite3.connect("school.db") as connection:
        cursor = connection.cursor()
        columns = ', '.join(values.keys())
        placeholders = ', '.join(['?'] * len(values))
        cursor.execute(f"INSERT INTO {table} ({columns}) VALUES ({placeholders})", tuple(values.values()))
        connection.commit()
```

In this function, we are inserting data into a specified table in the database. The **values means that we can pass a dynamic set of key-value pairs, where the keys represent the column names, and the values represent the data we want to insert into those columns.

- First, we create a connection to the database and get a cursor to execute SQL commands.
- We then create a string of column names (from the keys of the values dictionary) and a string of placeholders (question marks) that will correspond to each value.
- We execute the INSERT INTO SQL command, inserting the values into the appropriate columns of the table.
- Finally, we commit the transaction to save the changes.

```
def get_id(table, name, column):
    with sqlite3.connect("school.db") as connection:
        cursor = connection.cursor()
        if table == "students":
            cursor.execute("SELECT student_id FROM students WHERE student_name = ?", (name,))
        elif table == "courses":
            cursor.execute("SELECT course_id FROM courses WHERE course_name = ?", (name,))
        else:
            return None
        result = cursor.fetchone()
        return result[0] if result else None
```

Next, we have to retrieve the ID of either a student or a course based on the name provided. We decide which table to query based on the table name provided to the function.

- If the table is "students", we query the students table to fetch the student_id where the student_name matches the provided name.
- If the table is "courses", we query the courses table to fetch the course_id where the course_name matches the provided name.
- If the table is something else, we return None, indicating no valid ID could be found.

```
def enroll_student_in_course(student_name, course_name):
    student_id = get_id('students', student_name, 'student')
    course_id = get_id('courses', course_name, 'course')

    if student_id and course_id:
        with sqlite3.connect("school.db") as connection:
            cursor = connection.cursor()
            cursor.execute("INSERT OR IGNORE INTO student_courses (student_id, course_id) VALUES (?, ?)", (student_id, course_id))
            connection.commit()
            print(f"Student '{student_name}' enrolled in course '{course_name}' successfully.")
    else:
        print(f"Error: student or course not found.")
```

Next, we have to create a function where we can enroll a student in a course. We first need to find the IDs for both the student and the course using the get_id function.

- First, we fetch the student_id for the student using the get_id function, passing "students" as the table and the student's name.
- Similarly, we fetch the course_id for the course.
- Once we have both IDs, we check if both IDs are valid.
- If both IDs are valid, we insert a new record into the student_courses table to link the student to the course using the student_id and course_id.
- If either the student or the course is not found, we print an error message indicating that enrollment couldn't be completed.

```
def get_student_courses(student_name):
    student_id = get_id('students', student_name, 'student')
    if student_id:
        with sqlite3.connect("school.db") as connection:
            cursor = connection.cursor()
            cursor.execute("""
                SELECT courses.course_name
                FROM student_courses
                JOIN courses ON student_courses.course_id = courses.course_id
                WHERE student_courses.student_id = ?
            """, (student_id,))
            courses = cursor.fetchall()
            return [course[0] for course in courses]
    return []
```

Finally, we retrieve all the courses a student is enrolled in.

- We first get the student_id for the student using the get_id function, passing "students" as the table and the student's name.

- If we successfully retrieve a valid student_id, we proceed to query the student_courses table to find all courses linked to that student ID.
- We join the student_courses table with the courses table on the course_id to fetch the course_name.
- Finally, we return a list of course names that the student is enrolled in.

```
def main():
    create_database()

    while True:
        print("\nMenu:")
        print("1. Add a new course")
        print("2. Add a new student")
        print("3. Enroll a student in a course")
        print("4. List courses for a student")
        print("5. Exit")

        choice = input("Choose an option: ")

        if choice == "1":
            course_name = input("Enter course name: ")
            teacher_id = input("Enter teacher ID (optional, press Enter to skip): ") or None
            insert_data("courses", course_name=course_name, teacher_id=teacher_id)
            print("Course added successfully.")

        elif choice == "2":
            student_name = input("Enter student name: ")
            insert_data("students", student_name=student_name)
            print(f"Student '{student_name}' added successfully.")

        elif choice == "3":
            student_name = input("Enter student name: ")
            course_name = input("Enter course name: ")
            enroll_student_in_course(student_name, course_name)

        elif choice == "4":
            student_name = input("Enter student name: ")
            courses = get_student_courses(student_name)
            if courses:
                print("Courses enrolled:", ", ".join(courses))
            else:
                print("No courses found for this student.")

        elif choice == "5":
            print("Exiting program.")
            break

        else:
            print("Invalid option. Please try again.")
```

For the correct execution of the code we create a main function. We start by initializing the database using create_database(), ensuring that all necessary tables (like students, courses, and enrollments) are properly set up. Then, we enter an infinite loop, displaying a menu that allows us to interact with the school management system.

The menu presents us with five options: adding a new course, adding a new student, enrolling a student in a course, listing the courses a student is enrolled in, and exiting the program. For each option, we prompt the user to provide relevant information (such as course names, student names, or enrollment details) and call the appropriate functions (`insert_course`, `insert_student`, `enroll_student_in_course`, and `get_student_courses`) to perform the actions. After each operation, we provide feedback, like confirming the addition of a course or displaying the list of courses a student is enrolled in.

We continue looping until we choose the option to exit the program. If we enter an invalid choice, the system prompts us to try again. Overall, the `main()` function acts as our user interface, allowing us to manage students, courses, and enrollments in an interactive and organized way.

We can verify the program's correct execution by testing each option with an example course, such as *Maths*, and an example student, like *Bea*. The output correctly displays the expected messages, confirming that the program functions as intended.

Output:

```
Menu:
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. List courses for a student
5. Exit
Choose an option: 1
Enter course name: Maths
Enter teacher ID (optional, press Enter to skip):
Course added successfully.

Menu:
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. List courses for a student
5. Exit
Choose an option: 2
Enter student name: Beatriz
Student 'Beatriz' added successfully.
```

Menu:

1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. List courses for a student
5. Exit

Choose an option: 3

Enter student name: Beatriz

Enter course name: Maths

Student 'Beatriz' enrolled in course 'Maths' successfully.

Menu:

1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. List courses for a student
5. Exit

Choose an option: 4

Enter student name: Beatriz

Courses enrolled: Maths